

CS Assignment: Algorithms Analysis

Exercise 2: E-commerce Platform Search Function

1. Understanding Asymptotic Notation

Big O notation is basically a way to measure how an algorithm's performance scales as the amount of input data grows. It gives us the "worst-case" scenario for how much time or memory it will take, which helps us pick the best approach for our code.

For search operations, the scenarios look like this:

- **Best-case:** We find the item on the very first try ($O(1)$ time).
- **Average-case:** The item is somewhere in the middle of our data.
- **Worst-case:** We have to check every single item and it's either at the very end or doesn't exist at all.

2 & 3. Setup and Implementation

Here is my Java implementation for the Product class and both search algorithms. I added the Comparable interface so we can easily sort the array before doing the binary search.

```
import java.util.Arrays;

class Product implements Comparable<Product> {
    private int productId;
    private String productName;
    private String category;

    public Product(int id, String name, String category) {
        this.productId = id;
        this.productName = name;
        this.category = category;
    }

    public int getProductId() {
        return productId;
    }

    @Override
```

```

    public String toString() {
        return A31515;">"ID: " + productId + A31515;">" | " + productName
+ A31515;">" (" + category + A31515;">");
    }

    // We need this so Arrays.sort() knows how to order the products
    @Override
    public int compareTo(Product other) {
        return Integer.compare(this.productId, other.productId);
    }
}

public class StoreSearch {

    // Basic linear search - checks one by one
    public static Product linearSearch(Product[] arr, int targetId) {
        for (Product p : arr) {
            if (p.getProductId() == targetId) {
                return p;
            }
        }
        return null;
    }

    // Binary search - much faster but needs sorted array
    public static Product binarySearch(Product[] arr, int targetId) {
        int left = 0;
        int right = arr.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            int midId = arr[mid].getProductId();

            if (midId == targetId) {
                return arr[mid];
            } else if (midId < targetId) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return null;
    }

    public static void main(String[] args) {

```

```

        Product[] inventory = {
            new Product(105, A31515;">"Wireless Mouse",
A31515;">"Electronics"),
            new Product(101, A31515;">"Mechanical Keyboard",
A31515;">"Electronics"),
            new Product(109, A31515;">"Desk Lamp", A31515;">"Home"),
            new Product(102, A31515;">"Coffee Mug", A31515;">"Kitchen"),
            new Product(107, A31515;">"Ergonomic Chair",
A31515;">"Furniture")
        };

        System.out.println(A31515;">"--- Testing Linear Search ---");
        Product p1 = linearSearch(inventory, 109);
        System.out.println(A31515;">"Found (109): " + (p1 != null ? p1 :
A31515;">"Not Found"));

        System.out.println(A31515;">"\n--- Testing Binary Search ---");
        Arrays.sort(inventory);
        System.out.println(A31515;">"(Sorted the array first)");

        Product p2 = binarySearch(inventory, 107);
        System.out.println(A31515;">"Found (107): " + (p2 != null ? p2 :
A31515;">"Not Found"));
    }
}

```

Output / Proof of Work

```

> java StoreSearch
--- Testing Linear Search ---
Found (109): ID: 109 | Desk Lamp (Home)

--- Testing Binary Search ---
(Sorted the array first)
Found (107): ID: 107 | Ergonomic Chair (Furniture)

```

4. Analysis

When comparing them, linear search takes $O(n)$ time because it iterates through every element. Binary search is much faster at $O(\log n)$ because it chops the search area in half each time (though it does require the list to be sorted first).

For an e-commerce platform, **Binary Search** is definitely the better choice. If a store has millions of products, a linear search would take way too long and cause lag for the user. Binary search can find an item out of a million in just about 20 steps.

Exercise 7: Financial Forecasting

1. Understanding Recursive Algorithms

Recursion is when a method calls itself to break down a bigger problem into smaller, repeating chunks. You just have to make sure you include a "base case" so it knows when to stop, otherwise it will run forever and crash.

2 & 3. Setup and Implementation

For this one, I wrote a standard recursive method to calculate the future value, and then an optimized one to prevent the call stack from getting too deep.

```
public class FinanceForecaster {

    // Standard recursive way to calculate future value
    public static double calcFutureValue(double balance, double rate, int
years) {
        // Base case: no years left
        if (years == 0) {
            return balance;
        }
        // Add a year's worth of interest and call again
        return calcFutureValue(balance * (1 + rate), rate, years - 1);
    }

    // Optimized way using fast exponentiation so we don't blow up the
call stack
    public static double optimizedFutureValue(double balance, double rate,
int years) {
        return balance * power(1 + rate, years);
    }

    // Helper for the optimized version
    private static double power(double base, int exp) {
        if (exp == 0) return 1.0;

        double half = power(base, exp / 2);
```

```

        if (exp % 2 == 0) {
            return half * half;
        } else {
            return base * half * half;
        }
    }

    public static void main(String[] args) {
        double startAmount = 10000.00;
        double growth = 0.05;
        int years = 10;

        System.out.println(A31515;">"--- Normal Recursion ---");
        double fv1 = calcFutureValue(startAmount, growth, years);
        System.out.printf(A31515;">"Value after 10 years: $%.2f%n", fv1);

        System.out.println(A31515;">"\n--- Optimized ---");
        double fv2 = optimizedFutureValue(startAmount, growth, years);
        System.out.printf(A31515;">"Value after 10 years: $%.2f%n", fv2);
    }
}

```

Output / Proof of Work

```

> java FinanceForecaster
--- Normal Recursion ---
Value after 10 years: $16288.95

--- Optimized ---
Value after 10 years: $16288.95

```

4. Analysis

My standard recursive algorithm has a time complexity of $O(n)$ because it makes one call for every year we want to forecast. The issue with this is that if we try to forecast hundreds of years, it uses up a lot of memory on the call stack and could cause a StackOverflow error.

To optimize this, I used a divide-and-conquer approach (fast exponentiation) for the math. Instead of multiplying step-by-step, it halves the exponent each time. This brings the time complexity down to $O(\log n)$, making it way more efficient and safer for large numbers.